should all start with a lowercase letter or an underscore (Ruby itself has a couple of methods that start with a capital letter, but in general this is not something to do in your own code).

Global variables are prefixed with a dollar sign, $, and instance variables begin with an "at" sign, @. Class variables start with two "at" signs, @@. Although we talk about global and class variables here for completeness, you'll find they are rarely used in Ruby programs. There's a lot of evidence that global variables make programs harder to maintain. Class variables are not as dangerous as global variables, but they are still tricky to use safely—it's just that people tend not to use them much, because there are often easier ways to get similar functionality. Finally, class names, module names, and other constants must start with an uppercase letter. Samples of different names are given in Table 1, Example variable, class, and constant names, on page 23.

Following this initial character, a name can contain any combination of letters, digits, and underscores, with the exception that the character following an @ sign may not be a digit. However, by convention, multiword instance variables are written with underscores between the words, like first_name or zip_code, and multiword class names are written in MixedCase (sometimes called CamelCase) with each word capitalized, like FirstName or ZipCode. Constant names are written in all caps, with words separated by underscores, like FIRST_NAME or ZIP_CODE. Method names may end with the characters ?, !, and =.

Arrays and Hashes

Ruby provides a few different ways to combine objects into collections. Most of the time, you'll use two of them: Arrays and Hashes. An Array is a linear list of objects, you retrieve them via their index, which is the number of their place in the array, starting at zero for the first slot. A Hash is an association, meaning it is a key/value store where each value has an arbitrary key, and you retrieve the value via that key. Both arrays and hashes grow as needed to hold new elements. Any particular array or hash can hold objects of differing types; you can have an array containing an integer, then a string, then a floating-point number, as we'll see in a minute.

You can create and initialize a new array object using an *array literal*—a set of elements between square brackets. Given an array object, you can access individual elements by supplying an index between square brackets, as the next example shows. Note that Ruby array indices start at zero.

```
a = [1, 'cat', 3.14]   # array with three elements
puts "The first element is #{a[0]}"
 # set the third element
a[2] = nil
puts "The array is now #{a.inspect}"
```

produces:

```
The first element is 1
The array is now [1, "cat", nil]
```

You may have noticed that we used the special value nil in this example. In many languages, the concept of *nil* (or *null*) means "no object." In Ruby, that's not the case; nil is an object, just like any other. It's just an object that represents the concept of nothing. Anyway, let's get back to arrays and hashes.